Vendor Invoice Checker

Simple use case and demo guide

One-line explanation: Upload a vendor invoice PDF; local AI reads its text, organizes invoice details, and flags items for a person to review.

Why this demo exists

Finance and purchasing teams handle many vendor invoices. This demo shows how AI can help someone find key details—such as the vendor, invoice number, date, purchase order, tax, and total—without reading every field manually. It is a demonstration, not an accounting, fraud-detection, or compliance approval system.

What model does it use?

Qwen 2.5, running locally through **Ollama** on the project owner's Mac. The installed Ollama model is `qwen2.5:latest` (7.6B parameters, Q4_K_M quantization). The app does not call an OpenAI or Gemini cloud AI API. The Mac must stay awake with Ollama and Streamlit running for the share link to work.

How the app works

1. **Upload:** A person chooses a PDF in the Streamlit page.
2. **Read:** `pypdf` extracts selectable text from the PDF pages.
3. **Analyze:** The app sends that text to Qwen 2.5 through Ollama on the Mac.
4. **Structure:** The model returns invoice fields in JSON format.
5. **Review:** The page displays the extracted data and a Valid or Flagged message for a person to check.

Scanned image-only PDFs may not contain selectable text. The model can also misunderstand a document, so a person must verify every result. “Valid” is a suggestion, not proof that an invoice is correct.

Which file does what?

File or folder	Simple explanation
`app.py`	Front end: shows the page, accepts a PDF, displays results, and warns testers to use sample files.
`engine.py`	Processing: extracts PDF text and asks local Qwen 2.5 for structured invoice data.
`requirements.txt`	Lists the Python packages needed by the app.
`.gitignore`	Prevents local environments, temporary PDFs, and caches from being added to Git.
`README.md`	Setup, use, and public demo instructions.
`venv/`	This Mac's Python environment; not application source code to share.

Technical and UI one-liners for a video

- **Front end / `app.py` :** “The Streamlit front end lets a user upload an invoice and see the extracted details and review status.”
- **PDF reading / `engine.py` :** “The processing code reads selectable text from each page of the uploaded PDF.”
- **AI analysis / Ollama:** “The extracted text is sent to Qwen 2.5 running locally on the Mac, which returns invoice fields as JSON.”
- **Results / `app.py` :** “The page displays the model's fields and flags the invoice for a person to review.”
- **Dependencies / `requirements.txt` :** “This file lists the Python packages required to run the demo.”
- **Sharing / Cloudflare Tunnel:** “A temporary HTTPS tunnel lets remote testers reach the Streamlit page on this Mac while the Mac and both apps stay running.”

Start and share the demo

Start Ollama and Streamlit as described in `README.md` . In a second Terminal window, run:

```
cloudflared tunnel --url http://127.0.0.1:8501
```

Share the `https://...trycloudflare.com` link printed in Terminal. Keep the Terminal window, Ollama, Streamlit, and Mac running. Stop the share link with **Control+C** in the tunnel window.

Privacy: This is an open public demo link with no login. Anyone who has the link can upload files while the tunnel is on. Use only sample, non-sensitive invoices. The model runs on the Mac, but network traffic goes through Cloudflare's tunnel. This is not a production deployment.